

S

ShopEZ

Backend Documentation

Angular 19

.NET 9 (C#)

SQL Server

YARP Gateway

Docker

JWT Auth

xUnit + Jasmine

Project Details

Trainee

Yadavali Jayasree

Batch

B3_Angular

Trainer

Narasimha Rao

Email

yadavalijayasree9@gmail.com

Program

UpGrad CTS Pre-Assessment Hiring

Submission

May 2026

1. Project Overview

ShopEZ's backend is a microservices application built on .NET. It is the server-side half of the UpGrad CTS Capstone Project. Each business domain has its own independently deployable service with its own database, communicating with the Angular frontend through a single YARP API Gateway.

The architecture follows the Repository Pattern — separating database access from business logic — and uses a shared class library (ShopEZ_Shared_Lib) to avoid duplicating DTOs, JWT utilities, and middleware across services.

Component	Value
.NET Platform	ASP.NET Core Web API
Architecture	Microservices + API Gateway (YARP)
Language	C#
ORM	Entity Framework Core + Dapper (hybrid)
Database	SQL Server (LocalDB / SQLEXPRESS)
Auth	JWT Bearer (HMAC-SHA256)
Password Hashing	BCrypt.Net
Testing	xUnit + Moq — 83 tests, 0 failures
Services	Auth :7001 Products :7002 Cart :7003 Orders :7004

1.1 Microservices Overview

Service	Port	Database	Responsibility
API Gateway	7000	—	YARP reverse proxy, JWT validation, CORS
AuthService	7001	ShopEZ_AuthDb	User registration, login, JWT issuance, user listing
ProductService	7002	ShopEZ_ProductDb	Product & category CRUD, image upload, search, pagination
CartService	7003	ShopEZ_CartDb	Per-user cart item persistence, add/update/remove/clear
OrderService	7004	ShopEZ_OrderDb	Order placement, history, admin status management

1.2 Shared Library — ShopEZ_Shared_Lib

This class library is referenced by all services and contains:

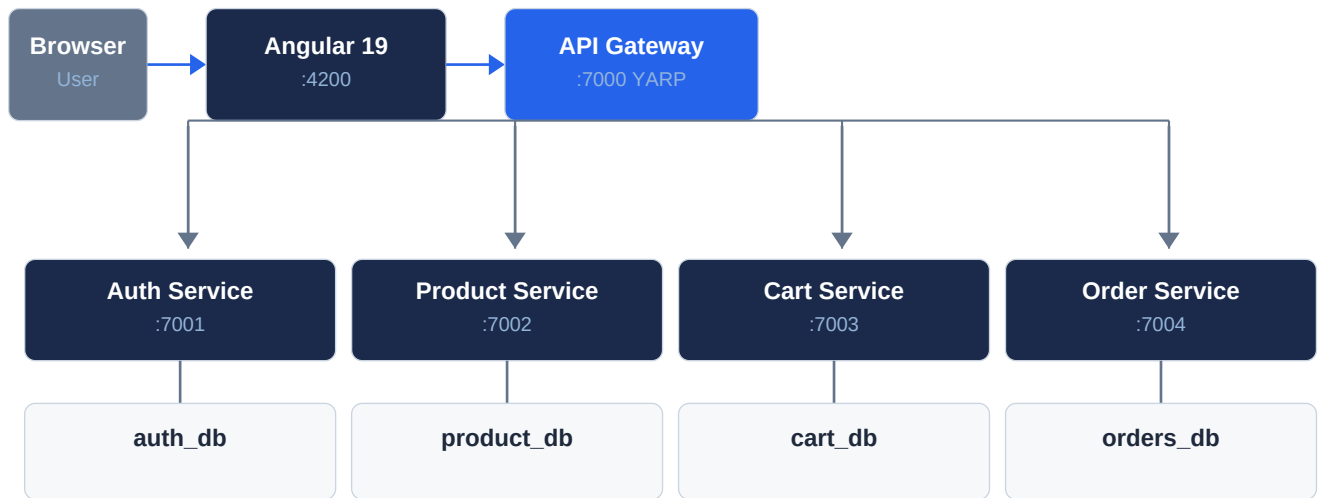
- DTOs: RegisterDTO, LoginDTO, AuthResponseDTO, UserDTO, AddToCartDTO, CartResponseDTO, CreateOrderDTO, OrderResponseDTO, CreateProductDTO, UpdateProductDTO, ProductResponseDTO,

PagedProductResponseDTO

- ApiResponse: Generic response envelope with Success, Message, Data, and Count fields
- JwtHelper: GenerateToken() and ValidateToken() using HMAC-SHA256 with shared secret
- ExceptionMiddleware: maps KeyNotFoundException → 404, UnauthorizedAccessException → 401, ArgumentException → 400, all others → 500

2. Architecture Deep-Dive

System Architecture Diagram



ShopEZ_Shared_Lib — DTOs · ApiResponse<T> · JwtHelper · ExceptionMiddleware (referenced by all 4 services)

Figure: ShopEZ Backend Architecture — Angular Frontend → YARP Gateway → Microservices → SQL Server Databases

2.1 API Gateway — YARP Reverse Proxy

The gateway is the single entry point for the Angular frontend. It uses Microsoft's YARP (Yet Another Reverse Proxy) configured in appsettings.json. JWT validation and CORS (allowing `http://localhost:4200`) are configured at the gateway level.

Route Pattern	Target Service	Port
/api/auth/**	AuthService	7001
/api/products/** and /api/categories/**	ProductService	7002
/api/cart/**	CartService	7003
/api/orders/**	OrderService	7004

2.2 Repository Pattern — 3-Layer Architecture

Each service defines repository interfaces and their concrete implementations, registered with ASP.NET Core's DI container as Scoped services. This makes each layer independently mockable in unit tests.

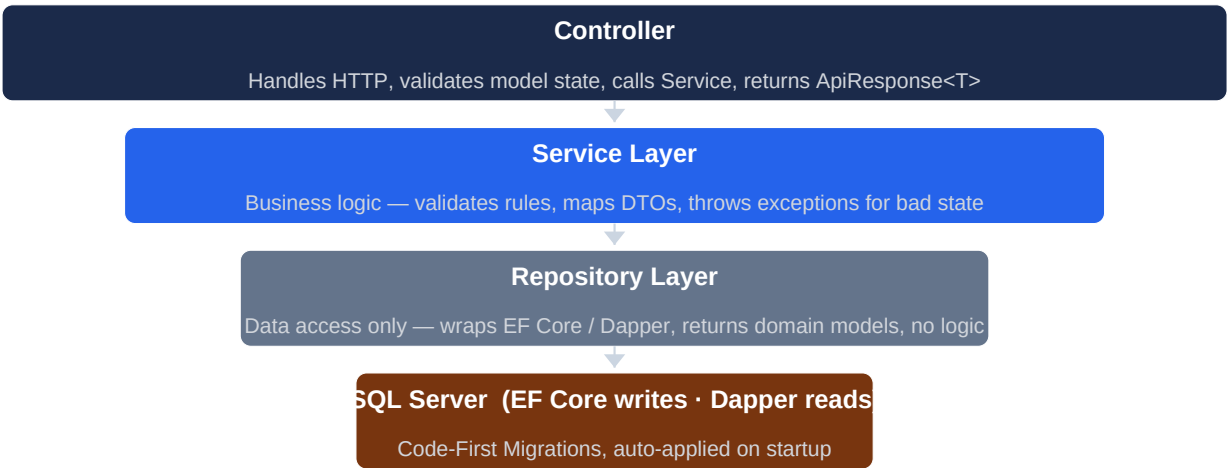


Figure: Repository Pattern — Controller → Service → Repository → Database

Layer	Responsibility
Controller	Receives HTTP request, validates model state, calls service, returns HTTP response
Service	Contains all business logic, maps domain models to/from DTOs
Repository	Handles all database operations, returns domain models

2.3 Hybrid ORM Strategy — EF Core + Dapper

ProductService deliberately uses two ORMs with a clear split in responsibility:

Operation	ORM	Reason
Reads (GET endpoints)	Dapper	Raw parameterised SQL mapped directly to DTOs. Significantly faster for read-heavy endpoints.
Writes (INSERT/UPDATE/DELETE)	EF Core	Type-safe change tracking, relationship management, and migration support.

2.4 JWT Authentication Flow

All five services share the same JWT configuration — same SecretKey, Issuer, and Audience — so a token issued by AuthService is trusted by the gateway and all microservices.

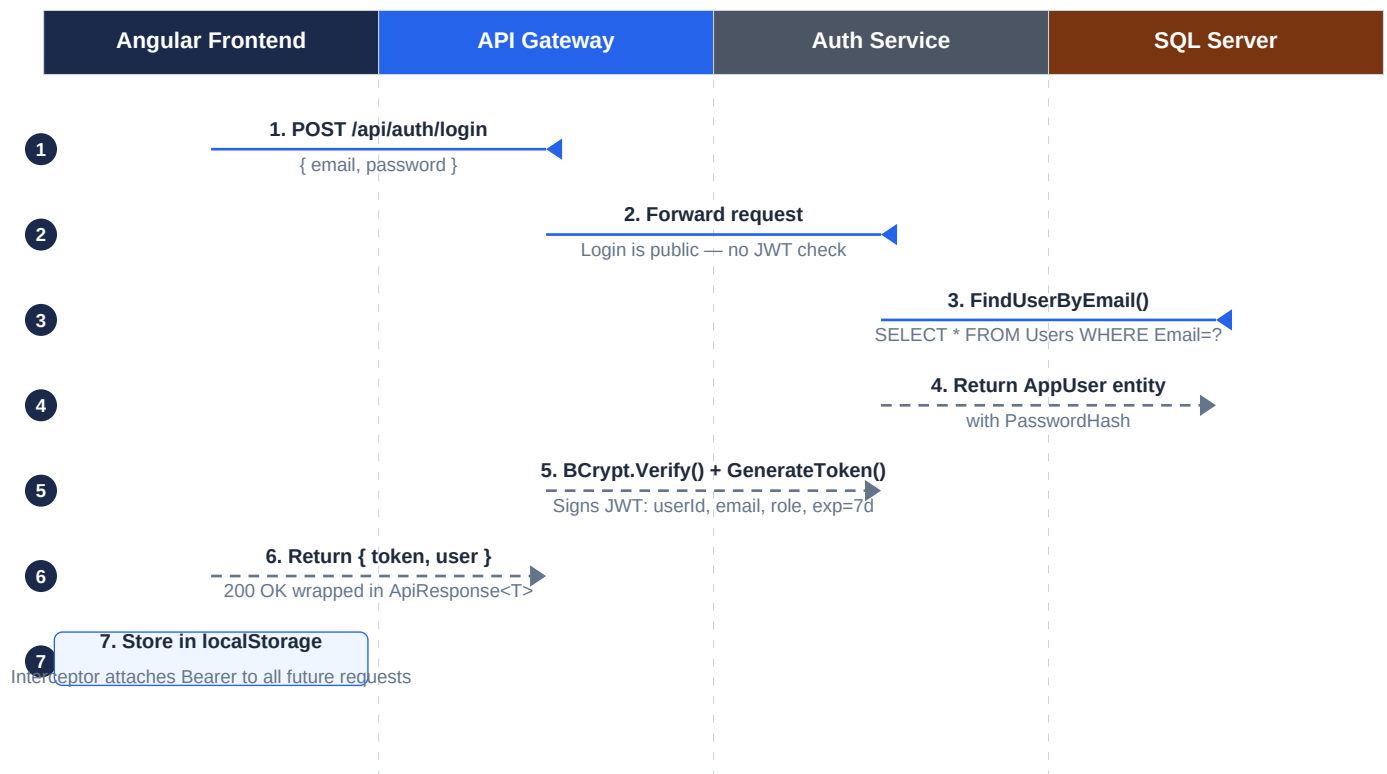


Figure: JWT Authentication Flow — Angular → Gateway → Auth Service → SQL Server → Token returned

JWT Claim	Value Stored
ClaimTypes.NameIdentifier	User ID (int)
ClaimTypes.Email	User email address
ClaimTypes.Role	CUSTOMER or ADMIN
JwtRegisteredClaimNames.Jti	Random GUID (prevents token replay attacks)
JwtRegisteredClaimNames.Iat	Unix timestamp of token issuance
Token Expiry	10,080 minutes (7 days)
Algorithm	HMAC-SHA256

3. API Endpoints

3.1 AuthService Endpoints

Endpoint	Method	Auth	Description
POST /api/auth/register	POST	Public	Creates CUSTOMER account. Validates email uniqueness, hashes password with BCrypt
POST /api/auth/login	POST	Public	Verifies credentials, checks IsActive flag, returns JWT + UserDTO
GET /api/auth/users	GET	ADMIN	Lists all users — used by admin dashboard stats
GET /api/auth/users/{id}	GET	ADMIN	Single user lookup by ID

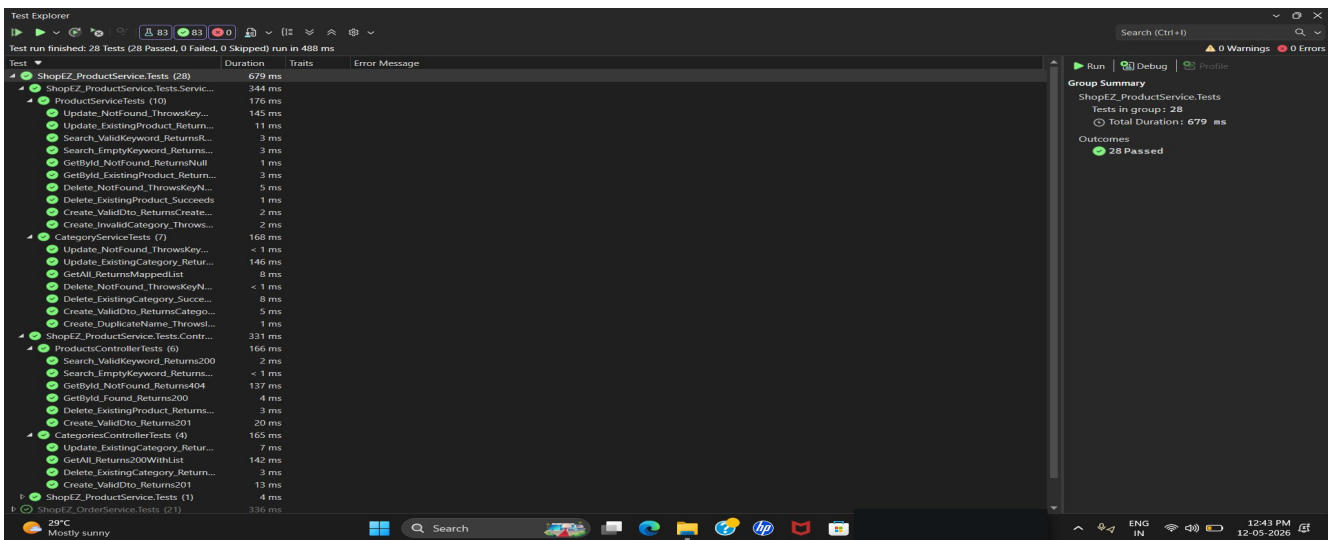


Figure: VS Test Explorer — ShopEZ Backend: 83 Tests Passed, 0 Failed (xUnit + Moq)

3.2 ProductService Endpoints

Endpoint	Method	Auth	Description
GET /api/products	GET	Public	Paginated list with optional keyword, categoryId, minPrice, maxPrice, sortBy
GET /api/products/{id}	GET	Public	Single product with category name (Dapper JOIN query)
POST /api/products	POST	ADMIN	Create with image upload (multipart/form-data)
PUT /api/products/{id}	PUT	ADMIN	Update fields and/or image
DELETE /api/products/{id}	DELETE	ADMIN	Soft delete — IsActive = false
GET /api/categories	GET	Public	All active categories

Endpoint	Method	Auth	Description
POST /api/categories	POST	ADMIN	Create category (name uniqueness enforced)
PUT /api/categories/{id}	PUT	ADMIN	Update name and description
DELETE /api/categories/{id}	DELETE	ADMIN	Blocked if category has products (HTTP 400)

3.3 CartService Endpoints

Endpoint	Method	Auth	Description
GET /api/cart	GET	User	All cart items for the authenticated user
POST /api/cart/items	POST	User	Add item — or increment qty if product already in cart
PUT /api/cart/items/{productId}	PUT	User	Update quantity for a specific product
DELETE /api/cart/items/{productId}	DELETE	User	Remove one product from cart
DELETE /api/cart	DELETE	User	Clear all items from user's cart

3.4 OrderService Endpoints

Endpoint	Method	Auth	Description
POST /api/orders	POST	User	Place order — creates Order + OrderItems in one transaction
GET /api/orders/my-orders	GET	User	Paginated order history for the authenticated user
GET /api/orders/{id}	GET	User/Admin	Single order — customers can only access their own (403 for others)
GET /api/orders/all-orders	GET	ADMIN	All orders across all users (paginated)
PATCH /api/orders/{id}/status	PATCH	ADMIN	Update order status

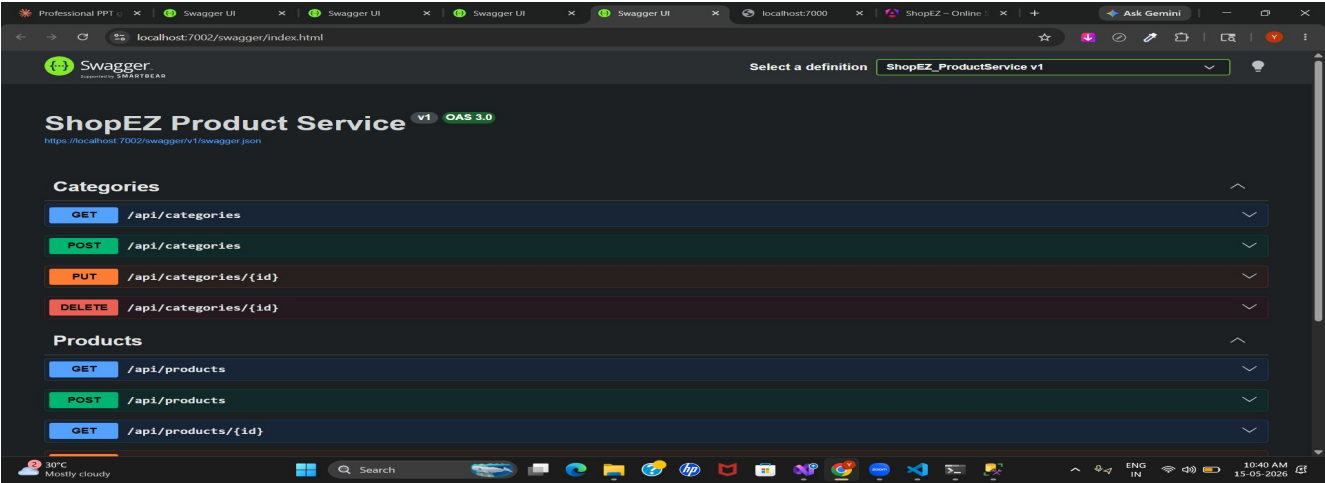


Figure: Swagger UI — ShopEZ Product Service (localhost:7002/swagger) — Categories and Products endpoints

4. Database Schema (SQL Server)

Each service has its own isolated SQL Server database. Services never share tables or query each other's databases directly — cross-service references are stored as plain integer IDs without foreign key constraints.

AppUser << entity >>	Product << entity >>	CartItem << entity >>	Order << entity >>
Id <i>int PK</i>	ProductId <i>int PK</i>	Id <i>int PK</i>	OrderId <i>int PK</i>
Name <i>nvarchar(100)</i>	Name <i>nvarchar(200)</i>	UserId <i>int</i>	UserId <i>int</i>
Email <i>nvarchar(200) UQ</i>	Price <i>decimal(10,2)</i>	ProductId <i>int</i>	OrderNumber <i>nvarchar(30)</i>
PasswordHash <i>nvarchar(max)</i>	Stock <i>int</i>	ProductName <i>nvarchar(200)</i>	TotalAmount <i>decimal(10,2)</i>
Role <i>nvarchar(20)</i>	ImageUrl <i>nvarchar(500)</i>	UnitPrice <i>decimal(10,2)</i>	Status <i>nvarchar(50)</i>
IsActive <i>bit</i>	IsActive <i>bit</i>	Quantity <i>int</i>	PaymentMethod <i>nvarchar(50)</i>
CreatedAt <i>datetime2</i>	CategoryId <i>int FK</i>	AddedAt <i>datetime2</i>	ShippingAddress <i>nvarchar(500)</i>

Figure: UML Class Diagram — 4 isolated entities across 4 databases (no cross-database foreign keys)

4.1 AppUser — ShopEZ_AuthDb

Column	Type	Constraints / Notes
Id	int (PK)	Identity, auto-increment
Name	nvarchar(100)	Required. Normalised: trimmed on save
Email	nvarchar(200)	Required. Unique index. Stored lowercase
PasswordHash	nvarchar(max)	BCrypt hash (default cost factor ~12)
Role	nvarchar(20)	Default: 'CUSTOMER'. Seeded ADMIN: 'ADMIN'
Phone	nvarchar(20)	Nullable
IsActive	bit	Default: 1. Login blocked if 0 (soft disable)
CreatedAt	datetime2	Default: GETUTCDATE()

4.2 Product & Category — ShopEZ_ProductDb

Column	Type	Notes
ProductId	int (PK)	Identity
Name	nvarchar(200)	Required
Description	nvarchar(2000)	Nullable

Column	Type	Notes
Price	decimal(10,2)	Required, range 0.01–999999.99
Stock	int	Default 0
ImageUrl	nvarchar(500)	Relative path: /images/products/.
IsActive	bit	Default 1 — soft delete sets this to 0
CategoryId	int (FK)	→ Categories.CategoryId, DeleteBehavior.Restrict

4.3 Order & OrderItem — ShopEZ_OrderDb

Column	Type	Notes
OrderId	int (PK)	Identity
UserId	int	JWT claim — no FK (cross-service isolation)
OrderNumber	nvarchar(30)	Unique index. Format: ORD-YYYYMMDD-XXXXXX
TotalAmount	decimal(10,2)	Calculated server-side — never trusted from client
Status	nvarchar(50)	Default: PENDING. Values: PENDING, CONFIRMED, SHIPPED, DELIVERED, CANCELLED
ShippingAddress	nvarchar(500)	Full address string from checkout form
PaymentMethod	nvarchar(50)	Default: CASH_ON_DELIVERY

4.4 CartItem — ShopEZ_CartDb

Column	Type	Notes
Id	int (PK)	Identity
UserId	int	No FK — cross-service integer reference
ProductId	int	No FK — cross-service integer reference
ProductName	nvarchar(200)	Snapshot stored at time of add to cart
UnitPrice	decimal(10,2)	Updated on re-add to sync with current product price
Quantity	int	Range 1–100

5. Key Implementation Details

5.1 Server-Side Pagination — Dapper

`ProductService.GetPagedAsync()` builds a dynamic SQL WHERE clause from optional filter parameters, then uses SQL Server's OFFSET/FETCH for true server-side pagination. Supported filters: keyword (LIKE on name/description), categoryId, minPrice, maxPrice, sortBy. All parameters are fully parameterised to prevent SQL injection.

5.2 Image Upload — ProductService

Product images are uploaded as multipart/form-data (IFormFile). The `SaveImageAsync()` method applies the following rules:

- Validates MIME type — accepts only image/jpeg, image/png, image/webp
- Enforces 5MB maximum file size
- Saves to `wwwroot/images/products/`. — GUID ensures no filename collisions
- Returns relative URL `/images/products/` stored in `Products.ImageUrl`
- Static file serving enabled via `app.UseStaticFiles()` in `Program.cs`

5.3 Order Placement — Business Logic

`OrderService.PlaceOrderAsync()` implements these rules before persisting to the database:

- Validates items list is not empty — throws `ArgumentException` if empty
- Calculates TotalAmount server-side: $\text{sum}(\text{UnitPrice} \times \text{Quantity})$ — never trusts the client total
- Generates unique order number: ORD-YYYYMMDD + 6-char uppercase GUID fragment
- Sets Status = 'PENDING' always — the client cannot set initial status
- Stores ImageUrl snapshot in each `OrderItem` — product thumbnails persist even if product is later deleted

5.4 Order Access Control

`GetById()` implements role-based access control in code. If the requesting user is not ADMIN and their `userId` does not match the order's `userId`, the endpoint returns HTTP 403 Forbidden.

5.5 Admin Seeder — Auto-Created Admin Account

Setting	Value
Admin Email	admin@shopez.com
Admin Password	Admin@123
Admin Name	Super Admin
Seeding Trigger	AuthService startup — only creates account if it does not already exist

6. Unit Testing

6.1 Results Summary — 83 Tests, 0 Failures

Framework: xUnit + Moq | Runner: Visual Studio Test Explorer. All repository interfaces are mocked using Moq — tests run without a real database and verify business logic in isolation.

Test Project	Tests	Status
ShopEZ_AuthService.Tests	18	✓ 18/18 Passed
ShopEZ_ProductService.Tests	28	✓ 28/28 Passed
ShopEZ_CartService.Tests	16	✓ 16/16 Passed
ShopEZ_OrderService.Tests	21	✓ 21/21 Passed
TOTAL	83	All Passed — 0 Failures

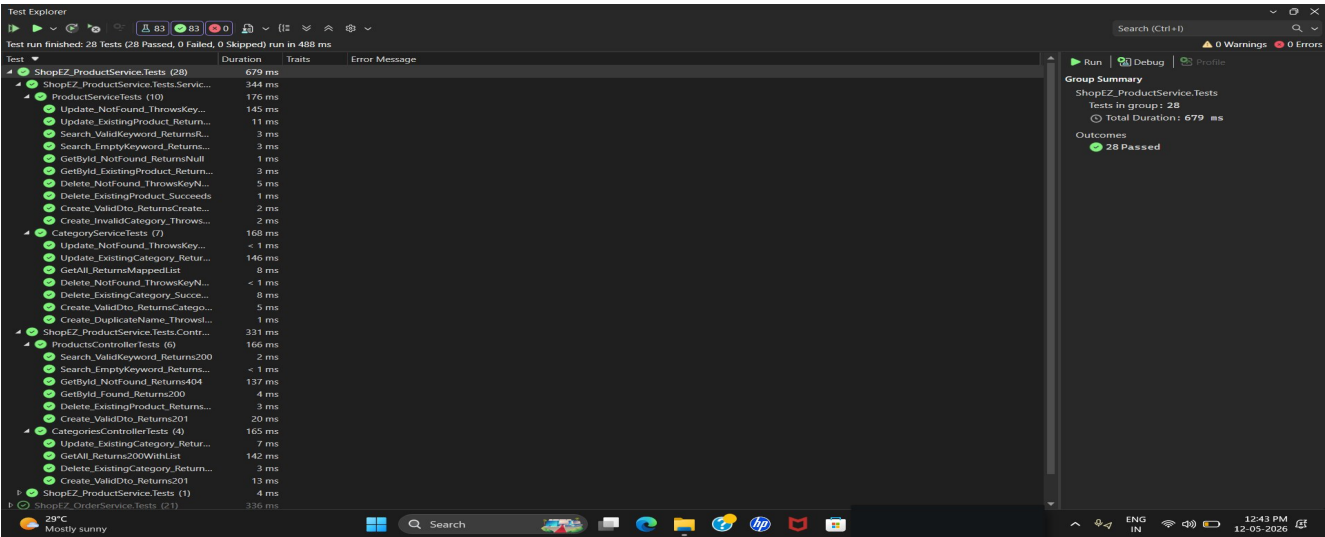


Figure: VS Test Explorer — All 83 xUnit tests passing across 4 test projects (Auth: 18, Product: 28, Cart: 16, Order: 21)

6.2 AuthService Tests — 18 Passed

Test Name	What It Verifies
Register_WithValidData	New user is created and JWT token is generated
Register_EmailAlreadyExists	Duplicate email throws ArgumentException
Register_TrimsNameAndLowerCases Email	Name is trimmed, email is lowercased before save
Login_WithCorrectCredentials	Valid credentials return AuthResponseDTO with token
Login_WithWrongPassword	BCrypt.Verify returns false → throws exception
Login_WithDeactivatedAccount	IsActive=false blocks login even with correct password

6.3 ProductService Tests — 28 Passed

Test Group	Key Tests
ProductServiceTests (10)	Create_ValidDto_ReturnsCreated, Create_InvalidCategory_Throws, Update_NotFound_ThrowsKeyNotFound, Delete_ExistingProduct_Succeeds, Search_ValidKeyword_ReturnsResults
CategoryServiceTests (7)	Create_ValidDto_ReturnsCategory, Create_DuplicateName_Throws, Update_ExistingCategory, GetAll_ReturnsMappedList
ProductsControllerTests (6)	Search_ValidKeyword_Returns200, GetById_Found_Returns200, GetById_NotFound_Returns404, Create_ValidDto_Returns201
CategoriesControllerTests (4)	GetAll_Returns200WithList, Create_ValidDto_Returns201, Update_ExistingCategory, Delete_ExistingCategory

6.4 CartService Tests — 16 Passed

Test Name	What It Verifies
AddItem_NewProduct_CreatesCartItem	New CartItem is inserted when product not in cart
AddItem_ExistingProduct_AccumulateQty	Quantity is accumulated when same product added again
UpdateItem_ExistingItem_SetsNewQty	Quantity is updated correctly
GetCart_TotalAmountIsCorrect	TotalAmount = sum(UnitPrice × Quantity) computed correctly
ClearCart_ClearsAllItems	All items removed from cart

6.5 OrderService Tests — 21 Passed

Test Name	What It Verifies
PlaceOrder_ValidDto_CreatesOrder	Order and items are persisted, status = PENDING
PlaceOrder_EmptyItems_ThrowsException	Empty items list throws ArgumentException
PlaceOrder_TotalAmountCalculated	Server calculates total correctly (UnitPrice × Qty sum)
PlaceOrder_OrderNumberFormat	Order number matches ORD-YYYYMMDD-XXXXXX pattern
PlaceOrder_InitialStatusIsPending	Status is always PENDING regardless of client input
GetById_OtherUsersOrder_Returns403	Non-owner customer gets 403 Forbidden
GetById_AdminCanViewAnyOrder	Admin user can access any order

7. API Response Format

All endpoints return a consistent ApiResponse envelope from the shared library:

```
// Success shape
{ "success": true, "message": "Login successful.", "data": { ... } }

// Error shape
{ "success": false, "message": "An account with this email already exists." }
```

HTTP Code	When Used
200 OK	Successful GET, PUT, login, register
201 Created	New product, new order created
400 Bad Request	Model validation failure or business rule violation
401 Unauthorized	Missing or invalid/expired JWT token
403 Forbidden	Valid token but wrong role, or accessing another user's data
404 Not Found	Resource does not exist (KeyNotFoundException → ExceptionMiddleware)
500 Internal Server Error	Unhandled exception caught by ExceptionMiddleware

8. How to Run the Backend

Step	Action
1. Prerequisites	Install .NET SDK and SQL Server (LocalDB or SQLEXPRESS)
2. Connection Strings	Edit appsettings.json in each service. Default: Server=localhost\SQLEXPRESS
3. Databases Auto-Created	Each service calls db.Database.Migrate() on startup — no manual setup needed
4. Admin Auto-Seeded	admin@shopez.com / Admin@123 created by Seeder on AuthService first run
5. Start All 5 Projects	Run in order: Gateway (7000), Auth (7001), Products (7002), Cart (7003), Orders (7004)
6. Swagger UI (Dev)	Each service exposes /swagger — e.g. https://localhost:7001/swagger
7. Run Tests	Visual Studio: Test Explorer → Run All. CLI: dotnet test in each test project

8.1 Shared JWT Configuration

The following JWT settings must be identical in all five services (appsettings.json):

Setting	Value
SecretKey	ShopEZSuperSecretKey2025ForJWTTokenGeneration!!
Issuer	ShopEZ
Audience	ShopEZ
ExpiryInMinutes	10080 (7 days)
Algorithm	HMAC-SHA256
ClockSkew	Zero — tokens expire exactly at the exp value

19. Docker — Containerization & Deployment

Every ShopEZ microservice has its own Dockerfile. All 7 containers (Angular, Gateway, Auth, Product, Cart, Order, SQL Server) are orchestrated by a single docker-compose.yml at the root of the repository.

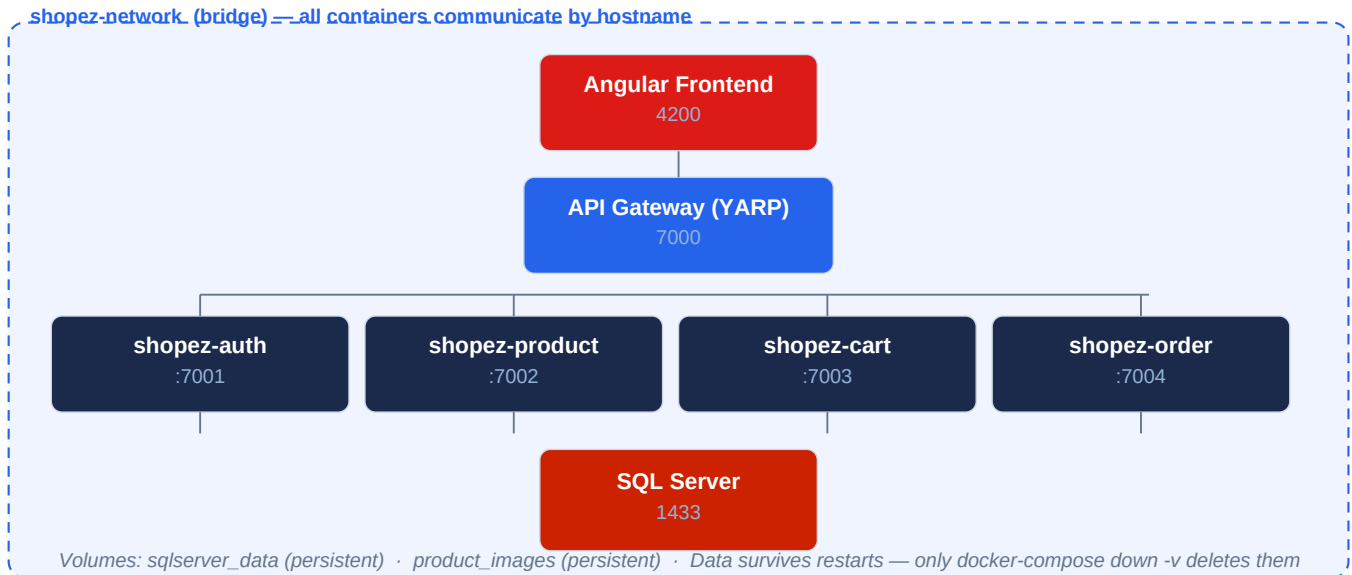


Figure: Docker Container Architecture — 7 containers on shopez-network bridge

19.1 Dockerfile Structure (Example — Auth Service)

```

FROM mcr.microsoft.com/dotnet/aspnet:9.0 AS base
WORKDIR /app
EXPOSE 80

FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build
WORKDIR /src
COPY ["ShopEZ_AuthService/ShopEZ_AuthService.csproj", "ShopEZ_AuthService/"]
COPY ["ShopEZ_Shared_Lib/ShopEZ_Shared_Lib.csproj", "ShopEZ_Shared_Lib/"]
RUN dotnet restore "ShopEZ_AuthService/ShopEZ_AuthService.csproj"
COPY . .
RUN dotnet build "ShopEZ_AuthService/ShopEZ_AuthService.csproj" -c Release -o /app/build

FROM build AS publish
RUN dotnet publish "ShopEZ_AuthService/ShopEZ_AuthService.csproj" -c Release -o /app/publish

FROM base AS final
WORKDIR /app
COPY --from=publish /app/publish .
ENTRYPOINT ["dotnet", "ShopEZ_AuthService.dll"]
  
```

19.2 Service Port Mapping

Container Name	Image	Host Port	Notes
shopez-angular	shopez-angular:latest	4200	Angular frontend — ng serve
shopez-gateway	shopez-gateway:latest	7000	YARP API Gateway — single entry point
shopez-auth	shopez-auth:latest	7001	Auth Service — JWT issuance
shopez-product	shopez-product:latest	7002	Product Service — also serves static images
shopez-cart	shopez-cart:latest	7003	Cart Service
shopez-order	shopez-order:latest	7004	Order Service
sqlserver	mcr.microsoft.com/mssql/server:2022	1433	SQL Server — all 4 databases

19.3 Key docker-compose.yml Configuration Points

- SQL Server healthcheck — all backend services use depends_on with condition: service_healthy so they wait until SQL Server is accepting connections before starting.
- Shared network — all containers communicate on the shopez-network bridge network using container names as hostnames (e.g., http://auth-service:80).
- Persistent volumes — sqlserver_data and product_images are named Docker volumes. Data survives container restarts and rebuilds. Only docker-compose down -v deletes them.
- Environment variables — JWT secrets, connection strings, and CORS origins are injected as environment variables. No secrets are hardcoded in source code.
- YARP Gateway routing — the API Gateway receives upstream service addresses via environment variables, e.g., ReverseProxy__Clusters__auth-cluster__Destinations__auth-service__Address.

19.4 Build & Run Commands

Command	Purpose
docker-compose up --build	Build images and start all containers
docker-compose up	Start all containers (no rebuild)
docker-compose down	Stop all containers (data preserved)
docker-compose down -v	Stop containers and delete volumes (data lost)
docker ps	List all running containers
docker logs shopez-auth	View logs for a specific service

9. Data Flow Diagram

9.1 Request Lifecycle — Step by Step

Step	Actor	Action
1	Angular AuthInterceptor	Reads JWT from localStorage, clones the HTTP request with Authorization: Bearer header
2	YARP API Gateway :7000	Receives the request, validates JWT signature and expiry using the shared HMAC-SHA256 secret, checks CORS origin (:4200)
3	YARP Route Matching	Matches route pattern (e.g. /api/orders/**) and proxies request to the correct microservice (:7004). Strips gateway prefix.
4	Controller	Extracts userId and role from JWT claims (ClaimTypes.NameIdentifier, ClaimTypes.Role). Validates model state.
5	Service Layer	Applies all business logic: validates items list, calculates TotalAmount server-side, enforces access control rules
6	Repository (EF Core / Dapper)	Persists data to the service's own SQL Server database in a single transaction
7	ExceptionMiddleware	Catches any unhandled exception — maps to correct HTTP status via ApiResponse envelope
8	Response	Controller wraps result in ApiResponse. YARP proxies response back to Angular.

9.2 Cross-Service Data Isolation

Service	Stores	Cross-Service Reference (no FK)
CartService	CartItem rows per user	UserId (from JWT claim), ProductId (client-provided integer)
OrderService	Order + OrderItem rows	UserId (from JWT claim), ProductId (snapshot — stored for history)
ProductService	Products + Categories	No cross-service references
AuthService	Users only	No cross-service references

10. Conclusion

ShopEZ's backend is a complete, production-quality microservices system that successfully demonstrates enterprise-level .NET development practices learned during the 3-month UpGrad CTS training program.

10.1 What Was Achieved

- Designed and implemented 4 microservices (Auth, Product, Cart, Order) plus a YARP API Gateway — 5 projects running as a single unified system
- Built a shared class library (ShopEZ_Shared_Lib) eliminating duplication of DTOs, JWT logic, and exception middleware across all services
- Applied the Repository Pattern with dependency injection throughout — all business logic is independently testable without a real database
- Implemented hybrid ORM strategy: Dapper for performance-critical reads, EF Core for type-safe writes
- Built JWT authentication with HMAC-SHA256 shared across all 5 services — role-based access control enforced at both gateway and controller level
- Wrote 83 xUnit tests with Moq mocking across 4 test projects — 0 failures
- Implemented secure image upload with MIME type validation, 5MB size limit, and GUID-based filenames
- Auto-seeded admin account on startup — no manual database steps required for a fresh environment

10.2 Challenges Overcome

Challenge	Solution Applied
Cross-service data without shared DB	Stored UserId and ProductId as plain integers in CartItem and OrderItem — no cross-database foreign keys
.NET MIME type with charset suffix	Stripped charset from Content-Type before MIME validation in SaveImageAsync()
Client total amount trust issue	TotalAmount always calculated server-side — client-submitted total is always ignored
Image persistence after product deletion	OrderItem stores ImageUrl as a snapshot at order time — soft delete does not break order history
JWT claims naming inconsistency	Used ClaimTypes constants (not string literals) across all services for consistent claim reading

11. Future Scope

Enhancement	Description
Real Payment Gateway	Integrate Razorpay or Stripe API in a new PaymentService
RabbitMQ / Event Bus	Cross-service events — e.g. OrderPlaced → ProductService decrements stock

Enhancement	Description
Redis Distributed Cache	Cache product list responses (TTL 5 min) to reduce Dapper query load
Refresh Tokens	Short-lived access tokens (15 min) + long-lived refresh tokens (7 days)
Kubernetes Deployment	K8s cluster with horizontal pod autoscaling, ingress controller replacing YARP
Centralised Logging (Serilog)	Serilog with Seq or Elasticsearch sink across all services
CQRS Pattern	Separate read and write models for OrderService and ProductService